Experiment No. 6

Title:

Write a code for optimal selection of items to maximize value within a weight constraint using bottom-up and top-down approaches.

Aim:

The aim of this program is to solve the 0/1 Knapsack problem using both bottom-up (iterative) and top-down (recursive with memoization) dynamic programming approaches. The goal is to determine the optimal selection of items to maximize the total value without exceeding the given weight capacity.

Theory:

0/1 Knapsack Problem

Given **N** items where each item has some weight and profit associated with it and also given a bag with capacity **W**, [i.e., the bag can hold at most **W** weight in it]. The task is to put the items into the bag such that the sum of profits associated with them is the maximum possible.

Note: The constraint here is we can either put an item completely into the bag or cannot put it at all [It is not possible to put a part of an item into the bag].

Examples:
Input: *N = 3, W = 4, profit[] = {1, 2, 3}, weight[] = {4, 5, 1}*
Output: *3*
Explanation: *There are two items which have weight less than or equal to 4. If we select the item with weight 4, the possible profit is 1. And if we select the item with weight 1, the possible profit is 3. So the maximum possible profit is 3. Note that we cannot put both the items with weight 4 and 1 together as the capacity of the bag is 4.*

Input: *N = 3, W = 3, profit[] = {1, 2, 3}, weight[] = {4, 5, 6}*
Output: *0*

Recursion Approach for 0/1 Knapsack Problem:
To solve the problem follow the below idea:

A simple solution is to consider all subsets of items and calculate the total weight and profit of all subsets. Consider the only subsets whose total weight is smaller than W. From all such subsets, pick the subset with maximum profit.

Optimal Substructure: *To consider all subsets of items, there can be two cases for every item.*

- ***Case 1:*** *The item is included in the optimal subset.*
- ***Case 2:*** *The item is not included in the optimal set.*

Follow the below steps to solve the problem:

The maximum value obtained from 'N' items is the max of the following two values.

- Case 1 (include the N^{th} item): Value of the N^{th} item plus maximum value obtained by remaining N-1 items and remaining weight i.e. (W-weight of the N^{th} item).
- Case 2 (exclude the N^{th} item): Maximum value obtained by N-1 items and W weight.

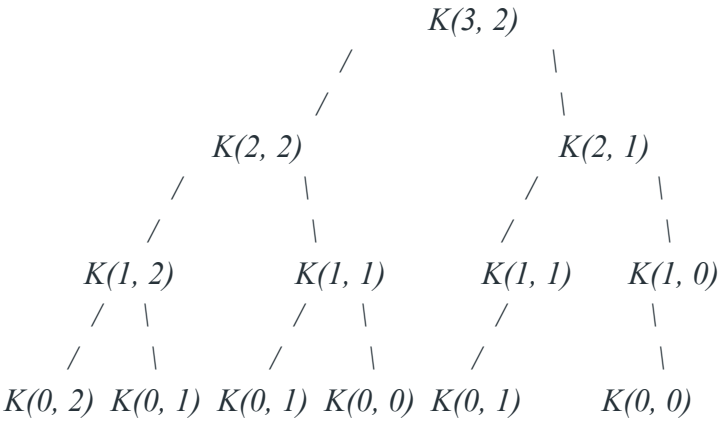
- If the weight of the 'Nth' item is greater than 'W', then the Nth item cannot be included and **Case 2** is the only possibility.

Dynamic Programming Approach for 0/1 Knapsack Problem

Memoization Approach for 0/1 Knapsack Problem:

Note: It should be noted that the above function using recursion computes the same subproblems again and again. See the following recursion tree, K(1, 1) is being evaluated twice.

*In the following recursion tree, **K()** refers to knapSack(). The two parameters indicated in the following recursion tree are n and W.*
The recursion tree is for following sample inputs.
weight[] = {1, 1, 1}, W = 2, profit[] = {10, 20, 30}



Recursion tree for Knapsack capacity 2 units and 3 items of 1 unit weight.

As there are repetitions of the same subproblem again and again we can implement the following idea to solve the problem.

If we get a subproblem the first time, we can solve this problem by creating a 2-D array that can store a particular state (n, w). Now if we come across the same state (n, w) again instead of calculating it in exponential complexity we can directly return its result stored in the table in constant time.

Pseudocode:

Bottom-Up Approach

```
def knapsack_bottom_up(values, weights, W):
    n = len(values)
    dp = [[0 for _ in range(W + 1)] for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(W + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(dp[i - 1][w], dp[i - 1][w - weights[i - 1]] + values[i - 1])
```

```
            else:
                dp[i][w] = dp[i - 1][w]

    return dp[n][W]

# Example usage:
values = [60, 100, 120]
weights = [10, 20, 30]
W = 50
print (knapsack_bottom_up(values, weights, W))  # Output: 220
```

Experiment 6 Title:

Experiment 6 Code:

Experiment 6 Output (Screenshot):

Bucket List Experiment Title:

Bucket List Experiment Code:

Bucket List Experiment Output (Screenshot):

Theory Questions (Handwritten):

Q1: Explain the difference between the bottom-up and top-down approaches in dynamic programming when solving the problem of optimal selection of items to maximize value within a weight constraint (e.g., the 0/1 Knapsack problem). How does each approach work, and what are the typical trade-offs in terms of implementation and performance?

Q2: Discuss the application of dynamic programming to the following problems and how it helps in optimizing their solutions:

i) **Shortest Paths**
ii) **Matrix Chain Multiplication**

iii) **Sequence Alignment**

Conclusion:

Both the bottom-up and top-down dynamic programming approaches effectively solve the knapsack problem, each with its own merits.